

AutomataGen: A Domain-Specific Language for On-Demand Generation and Closure-Based Expansion of Automata Theory Practice Problems

Nandigama Venkatesh

JNTU-GV College of Engineering, Vizianagaram

Vizianagaram, India

nvenkatesh.cse@jntugvce.edu.in

Kakaraparth V N P Siddhartha

CloudKeeper India Pvt. Ltd.

New Delhi, India

siddhartha.kakaraparth@cloudkeeper.com

Abstract

Automata theory is foundational to the undergraduate computer science curriculum, covering deterministic finite automata (DFA), pushdown automata (PDA), and Turing machines (TM). Mastering it requires practice constructing machines for precisely defined formal languages. Practice-oriented tools—most notably JFLAP and Automata Tutor—offer strong construction and simulation interfaces, but their problem pools are either absent (JFLAP: students bring their own problem) or limited to one machine type and derivation strategy (Automata Tutor’s automatic generation covers DFA only, from a seed problem). We present AutomataGen, a web-based practice platform for constructing DFA, PDA, and TM, where every problem is produced on demand by a custom domain-specific language (DSL). Instructors author a small set of parameterized specifications; at practice time, a student’s request triggers live instantiation, validation against four automatic quality guards, and expansion through a class-aware *closure-property generation* mechanism combining instantiated questions under union, intersection, difference, complement, and reversal. This mechanism respects each machine class’s true closure boundaries—restricting pushdown automata to the two operations under which context-free languages are closed, while deriving cross-class problems such as a PDA from a context-free/regular intersection—turning n specifications into up to $3\binom{n}{2} + 2n$ automatically validated problems without added authoring effort. Each problem carries machine-synthesized test cases, and DFA problems additionally carry a reference automaton enabling exact equivalence-based self-checking. We describe the DSL, the generation pipeline, a technical evaluation of its reliability and scale, and our plan for classroom deployment.

CCS Concepts

- **Theory of computation** → **Formal languages and automata theory**; • **Social and professional topics** → *Computing education*;
- **Applied computing** → *Interactive learning environments*.

Keywords

automata theory, domain-specific language, practice problem generation, computing education, formal languages

ACM Reference Format:

Nandigama Venkatesh and Kakaraparth V N P Siddhartha. 2026. AutomataGen: A Domain-Specific Language for On-Demand Generation and Closure-Based Expansion of Automata Theory Practice Problems. In *Proceedings of ACM COMPUTE Conference 2026 (COMPUTE 2026)*. ACM, New York, NY, USA, 5 pages.

1 Introduction

Mastering automata theory—a foundational component of the undergraduate computer science curriculum—requires students to move beyond theory and practice constructing machines that accept precisely defined formal languages. Interactive tools such as JFLAP [6] give students an excellent canvas for building and simulating automata, and Automata Tutor [1, 3] goes further by automatically generating an unbounded number of DFA construction problems of tunable difficulty. Yet the pool of problems a student can practice against remains limited: JFLAP supplies no problem bank at all, leaving an instructor to author, vary, and distribute problems by hand, while Automata Tutor’s automatic generation is specific to DFA construction, with PDA and TM problems in that system, to our knowledge, instructor-authored and fixed. This gap is why automata practicals traditionally reproduce the same handful of textbook exercises across cohorts and years, inviting rote memorization and answer-sharing rather than genuine skill practice.

We present AutomataGen, a web platform built around a single idea: instructor-authored specifications. In each session, a student’s request for a new problem triggers instantiation from the specification, validation against four automatic quality guards, and, where applicable, closure-property derivation—producing a fresh, validated question with its own test cases in seconds. The contributions in this paper are as follows.

- A DSL that expresses automata theory problem families uniformly across DFA, PDA, and TM (Automata Tutor’s generation covers DFA only), with counting, length, ordering, and structural-predicate primitives.
- A class-aware *closure-property generation* mechanism (union, intersection, difference, complement, reversal) that respects each machine class’s true closure properties, derives cross-class problems (e.g. a new PDA from a $\text{DFA} \cap \text{PDA}$ intersection), and expands n authored specifications into as many as $3\binom{n}{2} + 2n$ structurally distinct, automatically validated questions.
- A generation pipeline with four always-on quality guards (distinguishability, non-emptiness, non-triviality, test-case balance) that discards and retries degenerate instances automatically, validated by a technical evaluation of its reliability and scale.

2 Related Work

Tools such as JFLAP [6], Automatarium [2], and Flap.js [5] provide excellent visual environments for constructing and simulating state

machines. However, these tools are primarily designed as pedagogical sandboxes. They allow students to experiment but lack the infrastructure for automated question generation.

AutomataTutor [3, 4] is the closest prior system in spirit: its automatic-grading foundation [1] introduced MOSEL, a logic for describing regular languages, and an algorithm for generating an arbitrary number of DFA-construction problems at a tunable difficulty relative to a seed problem. Automata Tutor v3 [3] extended the system to additional problem types (regular expressions, context-free grammars, PDA, TM) and reported a large-scale usability survey. To the best of our knowledge its automatic-generation capability remains specific to DFA construction; PDA and TM assignments still require an instructor-authored reference automaton and question, with AutomataTutor limited to verifying submissions against it. AutomataGen differs by generating problems uniformly across all three machine types from a single DSL, and by deriving additional variety through closure properties.

3 System Architecture

3.1 Overall Architecture

Figure 1 illustrates the *AutomataGen* pipeline for question generation. A DSL specification (.dsl) is first processed by the *DSL Compiler*, which comprises four internal stages: a *lexer* that tokenizes the specification text against 61 distinct token types, a *recursive-descent parser* that builds a typed Abstract Syntax Tree (AST) from the token stream via roughly 30 mutually recursive parsing routines, a *free-variable solver* that resolves any free parameters and inequality chains declared in the specification into a consistent concrete instance, and a *renderer* that converts the resolved AST pattern into a natural-language question (English). This produces a natural-language question, a synthesized test-suite, and, for DFA-family questions, a reference automaton for exact equivalence-based self-checking. Students submit their constructed automata directly within the web platform, and the evaluator scores each submission against the generated test suite and, for DFA questions, the reference automaton via Hopcroft minimization and BFS-based product simulation, returning a score together with witness-string or counter-example feedback.

4 The AutomataGen DSL

4.1 The Domain-Specific Language and Instantiation

Figure 2 shows a representative DFA specification. An alphabet of size 2 is first sampled from either $\{a, b, c\}$ or $\{0, 1\}$; the *roles* block then binds the abstract role *A* to the first symbol of that sampled alphabet. The *parameters* block declares *k* as a free integer in $[2, 6]$, sampled once per instantiation; the *condition* block is evaluated to synthesize the automaton accepting the described language, and the *testing* block requests 200 test cases bounded to strings of length 50 or fewer. For example, sampling the window $\{a, b\}$ from $\{a, b, c\}$ (binding role *A* to *a*) with $k = 6$ resolves `count(A) % k == 0` to the natural-language question: “Construct a DFA over the alphabet $\{a, b\}$ that accepts all strings where the number of ‘a’ symbols is divisible by 6.” Marking `closure_allowed: true` makes this specification eligible to be combined with any other DFA specification over a

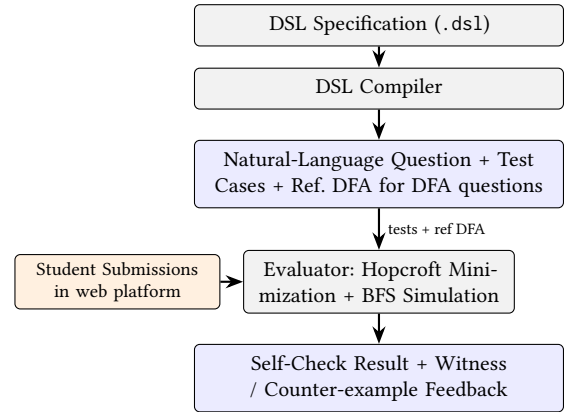


Figure 1: AutomataGen pipeline for question generation and self-assessment. The generator (first two stages) synthesizes a unique question, test suite, and reference machine on demand; the evaluator scores each submission against that generated artifact/test suite.

```

AUTOMATA DFA
template_id: DFA_MOD_COUNT
pattern_family: {counting}
difficulty: 1
description:
  Construct a DFA accepting strings where the
  number of A symbols is divisible by k.
alphabet:
  size: 2
  symbol_pools:{a,b,c}
              {0,1}
roles:{
  A <- alpha[0]
}
parameters:
  k in [2,6]
condition:
  count(A) % k == 0
testing:
  cases: 200
  max_length: 50
tags: {dfa,counting}
closure_allowed: true
END
  
```

Figure 2: Example DSL specification: a modular-counting DFA family.

compatible alphabet under all five closure operations described in Section 4.2.

A DSL specification is a plain-text .dsl file declaring: the target machine type (DFA/PDA/TM); an alphabet drawn from one of several symbol pools; role bindings (e.g. `A <- alpha[0]`) that keep specifications alphabet-agnostic; free parameters sampled from declared

ranges; a conjunction of conditions built from counting, length, ordering, and string-predicate primitives (e.g. `prefix_count(A) >= prefix_count(B)`, `palindrome`); and a testing block specifying how many test cases to synthesize and the maximum string length. At generation time the system samples a symbol window and parameter values, solves any free-variable inequalities (e.g. $M < N < P$) so that a consistent concrete instance exists within the length budget, and renders a natural-language question. The current specification library spans 55 DSL specifications (17 DFA, 16 PDA, 22 TM) covering pattern families such as modular counting, decimal-equivalent constraints, block ordering, balanced-bracket/Dyck conditions, and palindromes, at author-declared difficulty levels 0–5.

4.2 Closure-Property Generation and Validation

4.2.1 Class-Aware Structural Derivation. A primary novelty of *AutomataGen* is its ability to bypass manual authoring through the automated application of closure properties. Given instantiated base questions q_1 and q_2 over the same alphabet, the system synthesizes fundamentally new state-machine topologies by applying formal operations: Union ($L_1 \cup L_2$), Intersection ($L_1 \cap L_2$), Difference ($L_1 \setminus L_2$), Complement ($\overline{L_1}$), and Reversal (L_1^R). From n base specifications, this mechanism combinatorially yields up to $3\binom{n}{2} + 2n$ structurally distinct problems without requiring any additional authoring effort; distinctness itself is enforced by the same session-level *require_distinguishable* guard (Section 4.3) applied uniformly to base and closure-derived instances alike.

Crucially, the derivation engine is strict regarding formal language hierarchy boundaries. While regular languages (DFAs and NFAs) and recursive languages (TMs) support all five operations, the generator mathematically restricts context-free languages (PDAs) to Union and Reversal, explicitly preventing the generation of non-CFL problems via intersection or complementation — properties under which the CFL class is not closed. Furthermore, the system natively implements a construction by using a cross-class closure theorem: given a Regular-language base question q_{DFA} and a Context-Free base question q_{PDA} over the same alphabet, it derives a new PDA problem for $L_{\text{DFA}} \cap L_{\text{PDA}}$, since $L_{\text{CFL}} \cap L_{\text{Regular}}$ is context-free. This cross-machine construction is sampled alongside the within-class operations and is capped at 30% of the requested closure budget, keeping the derived-question mix representative of all five operations rather than dominated by the cross-class case.

4.2.2 Reference Construction and Validation. Each derived question is backed by a compositional reference construction: Boolean operations are realized as a product automaton over the two base machines’ state sets, and reversal reverses transitions and swaps the start/accept role, keeping DFA-family (and DFA-involving cross-class) closure questions exactly equivalence-checkable via the same Hopcroft-minimization pipeline used for base questions. Derived questions pass back through the four quality guards from Section 4.3, so a closure operation that collapses to a trivial or empty language is discarded rather than surfaced. A specification may opt out of closure combination entirely via a `closure_allowed: false` flag when its combinations would exceed the intended difficulty ceiling.

Table 1: Generation pipeline: technical evaluation

Metric	Value	Notes
DSL specifications authored	55	17 DFA, 16 PDA, 22 TM
Base instances requested	275	5 × 55 specifications
Base instances passing all guards	232 (84.4%)	auto-retried
Closure questions requested	20	5 op. types
Closure questions produced	16 (80.0%)	

4.3 Automatic Quality Guards

Generated problems, base or closure-derived, risk being mathematically degenerate — for example, an intersection that collapses to an empty language ($L_1 \cap L_2 = \emptyset$), or length constraints that leave a condition unsatisfiable within `max_length`. To keep instructors free of this burden, every instantiated problem is checked by four automatic guards before being offered to a student, executed in a cheapest-first cascading order that stops at the first failure to save compute cycles: *require_distinguishable* (checked against a session-level pattern registry to keep every offered problem unique), *require_non_empty* ($L \neq \emptyset$ within `max_length`), *require_non_trivial* ($L \neq \Sigma^*$, ruling out trivial accept-all machines), and *require_balanced* (at least three ACCEPT and three REJECT strings are reachable, a stricter check than mere non-triviality when `max_length` is tight). Problems that fail any guard are discarded and regeneration is retried (up to five times); specifications that still cannot produce a valid instance after retrying are flagged.

5 On-Demand Practice Workflow

A student selects a machine type and difficulty tier (easy/medium/hard, mapped to underlying specification difficulty levels) and requests a problem; the backend filters eligible specifications, instantiates one live, runs it through the quality guards, and—depending on machine type—attaches a small set of public example strings distinct from the hidden grading test cases plus, for DFA, a reference automaton for equivalence checking.

Figure 3 shows the practice interface for a generated DFA question. The left panel displays the natural-language question text, the sampled alphabet ($\{b, c\}$), the length bound, and a small set of public example test cases (labelled ACCEPT/REJECT) distinct from the hidden grading suite; the main canvas is a JFLAP-style construction surface where the student builds their automaton interactively.

6 Technical Evaluation

Because *AutomataGen* has not yet been deployed to a live cohort, we evaluate the pipeline’s reliability and scale rather than learning outcomes, requesting five instances per specification across all 55 specifications and 20 closure-derived questions end-to-end, with results in Table 1.

The 84.4% first-pass guard rate confirms the guards are doing real work—roughly one instance in six is discarded rather than shipped as an empty, trivial, or duplicate problem—while the pipeline completes each request within seconds, the latency needed for live,

[← Back](#) [DFA](#)

Question

Construct a DFA from the alphabet {*b*, *c*} that accepts all strings where the number of '*b*' symbols is divisible by 5.
(given: k=5)

Alphabet: {*b*, *c*}

Max length: 50

Machine: DFA

EXAMPLE TEST CASES

Accept: Input : bbbbbbcccccccccbbcbbbbcbccccccbb	
Accept: Input : bccccccccbbcbsccccccccccccccccc	
Reject: Input: bbbbbb	

Reject: Input: bbbbbb	 	

[Select](#)
 [State](#)
 [Transition](#)
 [Delete](#)
 [Fit view](#)
 [X Close](#)

+ Simulation Submission result

[VALENCE CHECK]
 {{ habit : {*b*, *c*} }
 Reference DFA : 5 states → 5 after minimisation
 Student DFA : 5 states → 6 after minimisation
X LANGUAGES DIFFER – witness: ‘bbbbb’ is accepted by one DFA but not the other.

Submit
 [Previous question](#)
 [Next question](#)

Figure 3: The on-demand practice interface: a generated DFA question (modular-counting family, $k = 5$) with its natural-language description, sampled alphabet, public example test cases, and the interactive construction canvas.

Table 2: Comparison with JFLAP and Automata Tutor

Capability	JFLAP	Automata Tutor	AutomataGen
Construction or simulation UI	Yes	Yes	Yes
Auto. problem generation	No	DFA only	DFA, PDA, TM
Per-question test cases	No	Internal	Yes, tunable
Closure-derived variants	No	No	Yes

on-demand use. The run’s 295 instances (275 base plus 20 closure-derived) already exceed the volume a single 30-student practice session would generate at one request per student, so the demonstrated throughput comfortably covers cohorts of that size. Table 2 compares against the two most closely related systems from Section 2.

7 Discussion and Conclusion

AutomataGen is intended for deployment in an undergraduate automata theory practical course, replacing a fixed problem sheet with session-based, non-repeating practice across DFA, PDA, and TM construction; we have not yet collected classroom data on learning outcomes, engagement, or instructor workload. Known limitations include: specification authoring requires DSL familiarity, and supporting a new condition operator requires extending the DSL Compiler (lexer, parser, and AST) rather than being expressible within existing specifications. We plan to (i) run a semester-long deployment with usage and outcome data, and (ii) explore automatic

DSL specification authoring assistance to lower the barrier for new instructors.

We presented AutomataGen, a DSL-driven web platform for automata theory practice that generates two kinds of questions on demand: base questions instantiated directly from instructor-authored DSL specifications, and class-aware closure-derived questions formed by combining pairs of instantiated specifications under union, intersection, difference, complement, and reversal, with four automatic quality guards validating the semantic well-formedness of every generated question, across DFA, PDA, and Turing machines. Each question, base or closure-derived, carries a machine-synthesized test suite, and DFA-family questions additionally carry a reference automaton enabling exact, equivalence-based self-checking; PDA and TM questions are self-checked against their generated test cases. The platform is live at <https://automata-gen.onrender.com/practice>, and its full source, including all 55 DSL specifications, is publicly available under the MIT license at <https://github.com/venkyjntu-git/AutomataGen>.

Acknowledgments

Portions of this manuscript’s prose and parts of the AutomataGen implementation were developed with the assistance of Claude (Anthropic); the authors reviewed all AI-assisted content and take full responsibility for its accuracy.

References

- [1] Rajeev Alur, Loris D’Antoni, Sumit Gulwani, Dileep Kini, and Mahesh Viswanathan. 2013. Automated Grading of DFA Constructions. In *Proceedings of the 23rd International Joint Conference on Artificial Intelligence (IJCAI’13)*. 1976–1982.
- [2] Automatarium Contributors. 2023. Automatarium: An Interactive Automata Simulator. <https://automatarium.tdib.xyz/>.
- [3] Loris D’Antoni, Martin Helfrich, Jan Kretínský, Emanuel Ramezant, and Maximilian Weininger. 2020. Automata Tutor v3. In *Computer Aided Verification (CAV 2020) (Lecture Notes in Computer Science, Vol. 12225)*. Springer, 3–14.

- [4] Loris D'Antoni, Matthew Weaver, Alexander Weinert, and Rajeev Alur. 2015. Automata Tutor and What We Learned from Building an Online Teaching Tool. *Bulletin of the EATCS* 3, 117 (2015), 144–158.
- [5] FLAPJS Contributors. 2022. Flap.js: A Web-Based Automata Simulator. <https://flapjs.web.app>.
- [6] Susan H. Rodger and Thomas W. Finley. 2006. *JFLAP: An Interactive Formal Languages and Automata Package*. Jones and Bartlett, Sudbury, MA.